BÁO CÁO BÀI TẬP

BTVN#3 — Dựng agent đặt vé máy bay bằng LangChain

Môn học: SE373— Agent Fundamentals

Sinh viên: Trần Đại Thắng

MSSV : 23521432

Ngày thực nghiệm: 01/10/2026 (Asia/Saigon)

Nội dung báo cáo

1. Mục tiêuvà phạm vi
2. Công nghệ và cấu trúc mã nguồn
3. Dữ liệu và tool mockup
4. Các lớp harness
5. Ba mẫu thiết kế
6. Thiết kế thực nghiệm
7. Kết quả thực tế
8. Phân tích trace và failure mode
9. Kiểm thử và tái lập
10. Giới hạn và hướng phát triển
11. Kết luận theo phạm vi thực nghiệm
12. Tài liệu tham khảo

Báo cáo được tạo từ mã nguồn Markdown trong repository bài tập. Kết quả thực nghiệm dùng model giả lập và môi trường mock; các giới hạn được nêu chi tiết trong phần 7 và 10.

1. Mục tiêu và phạm vi

Xây một agent đặt vé máy bay có đủ bốn thành phần harness theo đề: ràng buộc lưu bằng dữ liệu, tiêu chí hoàn thành kiểm bằng code, kiểm quyền trước hành động và bàn giao khi chưa hoàn thành. Cài ba cách tổ chức công việc: ReAct, Plan-then-Execute và mẫu Lai; đánh giá trên cùng điều kiện.

Bài toán cố định là đặt một vé một chiều cho một hành khách từ SGN đến DAD ngày 07/10/2026, cất cánh trước 12:00, tổng tiền không quá 2.000.000 VND. Hạn mức hành động tự động là 1.500.000 VND; vé vượt hạn mức này hoặc không hoàn được phải có phê duyệt. Người phê duyệt không được vượt trần cứng 2.000.000 VND bằng một câu đồng ý: muốn đổi trần phải tạo yêu cầu mới.

Backend và thanh toán đều là giả lập. Mặc định sử dụng MockDecisionModel, một lớp BaseChatModel có chính sách Python cố định. LangChain/LangGraph thực sự thực thi các graph và tool; không có LLM thật hoặc API hàng không trong lần đo này. Vì thế báo cáo đánh giá luồng điều khiển, các quy tắc harness và đánh đổi của ba cách tổ chức, chưa đánh giá năng lực suy luận LLM.

2. Công nghệ và cấu trúc mã nguồn

LangChaincung cấptoolcó schemavà create_agent choReAct. LangGraph cung cấp StateGraph, checkpoint trong bộ nhớ và interrupt để dừng/chờ người rồi tiếp tục. Pydantic kiểm dữ liệu yêu cầu, tham số tool và JSON kế hoạch.

File	Trách nhiệm
domain.py	BookingRequest ,schematool, violations, Scenario, MockAirline
harness.py	Harness, Limits,kiểm quyền/ràngbuộc/hoànthànhvàhandoff
agents.py	MockDecisionModel, SharedMiddleware, build_planned_agent, run_agent
btvn3_agent_dat_ve.py	Nhận tham số CLI, phê duyệt qua bàn phím, lưu kết quả
evaluate.py	Bộ chấm bên ngoài model, benchmark và xuất kết quả
test_btvn3.py	Kiểm thử cả ba mẫu và các hành vi sai

Tất cả đường thực thi tool đều đi qua Harness.execute. Ba mẫu không được tự gọi backend để bỏ qua kiểm quyền. Mỗi lượt chạy tạo backend, harness, model và checkpoint riêng, không dùng lại tồn kho hoặc bộ đếm từ lượt trước.

3. Dữ liệu và tool mockup

BookingRequest làPydanticmodel bất biến.Các trường gồm origin, destination, depart_date, depart_before, max_price, auto_approval_limit và approve_nonrefundable. Ngày/giờ được parse thành kiểu dữ liệu tương ứng, tiền lưu số nguyên VND.

Tool	Đầu vào chính	Quan sát / tác dụng
search_flights	Tuyến và ngày	cờ complete
check_seat	flight_id	Ghế, giá, thời gian, điều kiện vé mới nhất
hold_seat	flight_id, quoted_price	Tạo booking held, giảm tồn ghế
pay	booking_id	Ghi một khoản thanh toán giả lập, cập nhật booking
get_booking	booking_id	Đọc trạng thái và ràng buộc để kiểm chứng

`Tool trả {status: "ok", ...} hoặc {status: "error", error_code, hint, retryable}. Các lỗi có phân biệt`
hết ghế, đổi giá, ID không hợp lệ và timeout. Backend có thay đổi có kiểm soát ở thời điểm kiểm ghế hoặc giữ chỗ để mô phỏng dữ liệu tìm kiếm bị lỗi thời.

Giữ chỗ và thanh toán có tính idempotent trong một phiên: phát lại thao tác trên cùng booking không trừ ghế/thu tiền lần hai. Harness chặn đặt thêm một chuyến khác khi phiên đã có booking đang tồn tại.

4. Các lớp harness

4.1. Ràng buộc là dữ liệu

violations(flight, request) so tuyến, ngày, giờ và giá; dữ liệu thiếu hoặc sai kiểu cũng không đạt. Harness không cho model tự đổi tuyến/ngày trong search_flights và kiểm lại booking trước hành động ghi. Lời gọi giữ chỗ phải có flight ID từ tìm kiếm và báo giá từ check_seat. Nếu giá hoặc điều kiện vé đổi so với báo giá, trả STALE_QUOTE và yêu cầu kiểm lại.

4.2. Tiêu chí hoàn thành bằng code

Chỉ đặt trạng thái SUCCESS sau observation của `get_booking. is_complete` yêu cầu:

```
booking tồn tại và trùng bản ghi backend
AND status == confirmed
AND paid == True
AND không vi phạm tuyến/ngày/giờ/giá
AND sổ thanh toán ghi đúng số tiền của booking
```

Thông báo thành công từ model hoặc chỉ nhận kết quả của pay chưa đủ. Khi model dừng nhưng chưa có bằng chứng, harness trả UNVERIFIED_COMPLETION. Câu trả lời CLI được tạo từ trạng thái kiểm chứng, không lấy nguyên câu kết luận của model.

4.3. Kiểm quyền trước hành động

Trước giữ chỗ/thanh toán, harness kiểm ràng buộc cứng và quy tắc cần duyệt. Phê duyệt gắn với chữ ký dữ liệu gồm mã chuyến, tuyến, ngày, giờ, giá và điều kiện hoàn vé; không dùng một cờ approved=True chung cho mọi giao dịch. Payload xin duyệt ghi rõ phạm vi gồm giữ chỗ VÀ thanh toán cùng điều kiện, nên không cần hỏi lại khi pay nếu điều kiện không đổi.

interrupt(payload) được gọi trước tác dụng phụ. Người dùng có thể đồng ý/từ chối; Command(resume=...) trả quyết định cho graph. Khi node chạy lại sau resume, backend chưa bị thay đổi trước điểm dừng. Mã kiểm thử xác nhận chỉ có một giữ chỗ và một khoản thu. Nếu giá đổi, chữ ký cũ không còn duyệt được hành động mới.

4.4. Bàn giao và điều kiện dừng

Mỗi kết quả có handoff: lý do, yêu cầu ban đầu, các hành động đã thử, booking hiện có, tiền đã thu, các tác dụng phụ và câu hỏi cụ thể cho người nhận. Ví dụ khi polling hết giới hạn, bàn giao ghi rõ đã thanh toán nhưng chưa xác nhận; không tuyên bố đặt vé thành công.

Cơ chế	Cấu hình mặc định
Ngân sách tool	Tối đa 30 lần thực thi backend
Thời gian	60 giây, kiểm trước lời gọi model/tool
Phát hiện lặp	Lần xuất hiện thứ ba của cùng `(tool, args)` trong cửa sổ 8 hành động khi không tiến triển
Bế tắc	8 hành động không thêm sự kiện tiến triển
Polling xác nhận	`Tối đa 4 kết quả pending_confirmation`
Lập lại kế hoạch	Tối đa 8 lần

Cơ chế	Cấu hình mặc định
Chặn cuối cùng	`recursion_limit=300`, tính theo bước graph, không đồng nhất với số lời gọi model

Tiến triển được đo bằng các sự kiện hữu ích: đã tìm kiếm, đã kiểm tra một mã chuyến mới, đã giữ chỗ, đã trả tiền. Polling hợp lệ được theo dõi bằng bộ đếm riêng để không nhầm với lỗi lặp. Giới hạn thời gian được kiểm giữa các thao tác, không ngắt cưỡng bức một hàm đang chạy; thời gian chờ người duyệt cũng nằm trong giới hạn phiên hiện tại. Token được ghi khi provider có usage, nhưng bản này chưa cài trần token/tiền độc lập.

5. Ba mẫu thiết kế

5.1. ReAct

```
create_agent → model chọn 1 tool → harness kiểm/thực thi → observation
                        ↑                                       |
                        └──────── chưa dừng ────────────────────┘
```

SharedMiddleware đếm model call, kiểm lời gọi và chặn khi model tự nhận xong thiếu bằng chứng. Sau mỗi observation, model có thể đổi chuyến hoặc thử lại lỗi tạm thời. Mỗi lượt model chỉ được gọi một tool để không chạy đua giữa giữ chỗ và thanh toán.

5.2. Plan-then-Execute

```
model sinh Plan JSON → kiểm schema → executor lần lượt thực hiện các bước
                                   → kiểm chứng hoặc bàn giao khi bước hỏng
```

Chỉ gọi planner một lần. Kế hoạch dùng placeholder $candidate, $quote_price, $booking để gắn dữ liệu tool chưa có ở thời điểm lập kế hoạch. Executor chọn ứng viên hợp lệ rẻ nhất theo quy tắc cố định và giữ ứng viên đó trong kế hoạch. Một bước kiểm ghế bị lỗi/hết ghế hoặc không đáp ứng ràng buộc dẫn đến PLAN_FAILED. Polling là một phần của bước xác minh đã có trong kế hoạch, không phải lập lại kế hoạch.

Baseline này không có nhánh dự phòng hoặc retry tổng quát. Đây là một lựa chọn triển khai được công khai, không phải khẳng định mọi hệ thống Plan-then-Execute đều bắt buộc thất bại khi gặp biến động.

5.3. Mẫu Lai

```
planner → thực thi tối đa k=2 bước → xem observation → planner cập nhật phần việc còn lại
                                          |
                            thành công / chờ người / dừng lỗi
```

Sử dụng cùng planner và executor với Plan. Khác biệt là có cạnh quay lại planner sau hai bước hoặc ngay khi bước thất bại. Khi chuyến đầu hết ghế, observation đưa cho planner để chọn chuyến còn lại. Thông số --chunk-size điều khiển tần suất cập nhật; bộ đánh giá giữ cố định k=2.

5.4. Vai trò của model giả và phạm vi so sánh

Cả ba dùng cùng lớp model giả. Model chỉ nhận request và observation đã có, không nhận tên scenario, các sự kiện tương lai của backend hoặc nhãn kỳ vọng. Ở ReAct nó xuất action; ở Plan/Lai nó xuất danh sách action. Việc chọn vé rẻ nhất trong tập hợp lệ là chính sách Python cố định.

Prompt/hình thức đầu ra khác nhau là phần cần thiết để thể hiện kiến trúc. Chính sách chọn ứng viên, tool, dữ liệu và các kiểm tra harness được giữ chung. Kết quả của model giả chủ yếu phản ánh thiết kế điều khiển đã cài; không thể dùng các tỷ lệ này để dự báo tỷ lệ thành công của một LLM thật.

6. Thiết kế thực nghiệm

Scenario	Điều kiện	Có thể hoàn thành trong phiên?
normal	Có vé phù hợp	Có

Scenario	Điều kiện	Có thể hoàn thành trong phiên?
price_jump	Giá đầu tăng vượt trần	Có, dùng vé thứ hai
no_match	Tất cả vé vượt trần	Không
approval_yes	Vé cần duyệt, người đồng ý	Có
approval_no	Người từ chối	Không
approval_wait	Chưa có quyết định	Không
transient_error	Timeout một lần	Có
persistent_error	Timeout liên tục	Không
wrong_constraints	Có vé sai giờ/ngày/tuyến và vé đúng	Có
confirmation_delay	Cần đọc lại vài lần để confirmed	Có
hold_race	Mất ghế giữa kiểm tra và giữ chỗ	Có, dùng vé thứ hai

Mỗi scenario chạy cả ba mẫu ba lần: tổng 108 run. Các biến kiểm soát là request, backend ban đầu, model, giới hạn và quyết định người duyệt. Backend và model được khởi tạo mới mỗi run; sự kiện đổi giá/hết ghế gắn với hành động thay vì thời gian thực để đảm bảo tái lập. Có warm-up riêng từng graph, không đưa vào số liệu; thứ tự ba mẫu được xoay theo lượt đo.

evaluate.judge chấm ngoài model bằng trace, kết quả đọc lại và sổ thanh toán. Kiểm quyền theo đúng thứ tự trace: một phê duyệt xuất hiện sau hành động không thể hợp thức hóa hành động đó.

Các chỉ số:

- Tỷ lệ đặt thành công trên tất cả run: cho biết tỷ lệ trả được booking thật trong thế giới mock.
- Tỷ lệ đặt thành công trên nhóm có thể đặt: tránh tính từ chối/chờ người/không có vé là lỗi đặt vé của agent.
- Model/tool calls, thời gian thực thi trung bình và độ lệch chuẩn.
- Vi phạm ràng buộc, hành động chưa được duyệt, thành công không có bằng chứng.
- Bàn giao đủ trường: có lý do, danh sách đã thử, tác dụng phụ và câu hỏi khi chưa xong.

Nhãn `Scenario.expected` dùng cho regression test theo hành vi đã thiết kế. Bộ chấm chất lượng không tính `PLAN_FAILED` là đặt vé thành công chỉ vì test mong đợi trạng thái đó.

7. Kết quả thực tế

Nguồn: summary.json, runs.csv, bảng tự sinh và trace từng lượt.

Chỉ số	ReAct	Plan-then-Execute	Lai
Đặt thành công / tất cả	24/36 (66,7%)	12/36 (33,3%)	24/36 (66,7%)
Đặt thành công / run có thể đặt	24/24 (100%)	12/24 (50%)	24/24 (100%)
Model calls trung bình	4,92	1,00	2,75
Tool calls trung bình	4,67	3,17	4,67
Thời gian TB ± σ (ms)	15,57 ± 3,93 0	7,38 ± 1,41	11,70 ± 5,43 0
Vi phạm ràng buộc	0	0	0
Hành động chưa được duyệt	0	0	0
Báo thành công thiếu bằng chứng		0	

Chỉ số	ReAct	Plan-then-Execute	Lai
Bàn giao đủ trường theo bộ chấm	100%	100%	100%

Các run không thể đặt là no_match, approval_no, approval_wait, persistent_error. Dừng đúng ở những ca này không có nghĩa đã đặt vé. Tỷ lệ bàn giao chỉ đo đủ cấu trúc, chưa có người chấm mức dễ hiểu hoặc chất lượng nội dung. Thời gian gồm dựng graph và chạy model Python cục bộ; không chứa mạng hay thời gian suy luận LLM. σ ở bảng là độ lệch chuẩn trên toàn bộ 36 run mỗi mẫu, nên chứa cả khác biệt giữa scenario. Thực nghiệm lặp lại chỉ kiểm độ ổn định của luồng và đo runtime; không phải 36 mẫu suy luận độc lập. Token và chi phí API để null vì không có usage thật. Không suy ra phần trăm tiết kiệm tiền từ số model call.

Plan dùng ít tool một phần vì dừng sớm trong ca khó. Vì vậy không kết luận Plan hiệu quả nhất chỉ từ cột thời gian/tool calls. Trong scenario normal, cả ba đều hoàn thành với 5 tool calls; model calls lần lượt là 5, 1, 3. Lai đạt cùng số booking với ReAct trong bộ mock này và dùng ít lần ra quyết định hơn, nhưng có thêm trạng thái kế hoạch cần debug.

8. Phân tích trace và failure mode

8.1. Hết ghế sau tìm kiếm

Trong sold_out_plan_1.json, bước trace 4 check_seat(VN101) trả seats=0. Đây là điểm đầu tiên kế hoạch không thể tiếp tục. Trace 5 ghi plan_step_failed, trace 6 kết thúc PLAN_FAILED, không có thao tác giữ chỗ/thanh toán.

Trong sold_out_hybrid_1.json, cũng observation đó dẫn đến lần planner thứ hai ở trace 6. Agent kiểm VJ202, giữ ghế, thanh toán và đọc lại booking. ReAct xử lý tương tự bằng quyết định ngay ở lượt model kế tiếp.

8.2. Lặp lỗi công cụ

persistent_error_react_1.json có các trace 4, 6, 8 là cùng check_seat(VN101) trả TIMEOUT. Trace 9 dừng LOOP_DETECTED khi vẫn còn ngân sách. Phân loại nguyên nhân là lặp không tiến triển; không ghi chung thành hết lượt.

8.3. Người từ chối

Trong approval_no_hybrid_1.json, trace 7 yêu cầu phê duyệt, trace 8 là từ chối và trace 9 dừng. `bookings`, `charges`, side_effects đều rỗng. Việc thực thi được chặn trước thay đổi trạng thái backend.

8.4. Thành công giả và quên ràng buộc

Test dùng model trả "đã đặt thành công, mã BK999" ngay lập tức. Harness trả `UNVERIFIED_COMPLETION` vì không có observation xác nhận. Test khác cố giữ vé vượt 2 triệu dù đã gắn phê duyệt; harness chặn bằng CONSTRAINT_VIOLATION. Đây là các kiểm tra code thay vì chỉ nhắc model trong prompt.

9. Kiểm thử và tái lập

Đã chạy thành công 22 test method bằng unittest. Một method bao phủ ma trận 12 scenario × 3 mẫu; các method còn lại kiểm schema, tool không tồn tại, thiếu/đổi báo giá, thứ tự quyền, resume không thu tiền hai lần, từ chối/chờ người, giới hạn model/tool/thời gian, polling, bế tắc với các action khác nhau, không tạo booking thứ hai và không báo xong chỉ sau pay.

```
# Chạy tại thư mục cha chứa .venv và btvn3
.\.venv\Scripts\python.exe -X utf8 -m unittest discover -s btvn3 -p test_btvn3.py -v
.\.venv\Scripts\python.exe -X utf8 btvn3\evaluate.py --repeats 3
```

Môi trường ghi trong results/summary.json: Windows 11, Python 3.14.5, LangChain 1.4.3, langchain-core 1.6.6, LangGraph 1.2.13, Pydantic 2.13.5. Khi tái chạy, số lượt/trạng thái của mock phải giống; thời gian có thể thay đổi theo máy và tải hệ thống.

10. Giới hạn và hướng phát triển

Model giả không kiểm chứng chất lượng hiểu ngôn ngữ, lập kế hoạch hoặc tuân thủ tool calling của LLM thật. Request được nhập bằng JSON đã có cấu trúc, chưa có bước hiểu yêu cầu tiếng Việt tự do. Một lượt benchmark là một người/một chiều; chưa có vé khứ hồi, nhiều hành khách, hành lý hay hoàn tiền. Kho mock hữu hạn nên có thể kết luận không có vé dựa trên danh sách đầy đủ, điều này không áp dụng tự động cho API phân trang ngoài đời.

Checkpointer và backend lưu trong bộ nhớ. Resume hoạt động trong tiến trình hiện tại; JSON bàn giao không thay thế database/checkpoint bền vững qua tắt máy. Tác vụ đã thu tiền nhưng chưa confirmed sẽ được bàn giao, chưa tự động hủy/hoàn tiền.

Mã đã có tùy chọn --model provider:model-name qua init_chat_model, nhưng chưa chạy LLM thật do chưa có cấu hình provider. Bước tiếp theo là chọn một model có tool calling, giữ nguyên bộ test/harness, chạy nhiều lượt và ghi usage thật; sau đó mới so sánh chi phí tiền và chất lượng suy luận. Có thể bổ sung một baseline Plan có retry hữu hạn để tách ảnh hưởng của retry khỏi ảnh hưởng của replanning.

11. Kết luận theo phạm vi thực nghiệm

Trong bộ mock đã chạy, ReAct và Lai thíchnghiđược với hết ghế, tăng giá và lỗi tạm thời. Plan cố định hoàn thành nhanh với ít lượt model ở môi trường ổn định, nhưng bỏ lỡ bốn tình huống vẫn có thể đặt được. Harness dùng chung đảm bảo ba mẫu đều không đặt sai ràng buộc hoặc vượt quyền trong các test đã xây. Lựa chọn kiến trúc cần xét cả tỷ lệ hoàn thành, khả năng xử lý biến động và số lượt ra quyết định.